

Digital Image Processing
Image Restoration

Dr.Wafaa Shalash

Introduction

- Removal or reduction of degradations that have occurred during the acquisition of the images.
- Sources of degradation:
  - Noise
  - Out-of-focus blurring
  - Camera motion blurring

Model of Image Degradation

- In spatial domain:
  - Blurring by convolution:

$$g(x,y) = f(x,y) * h(x,y)$$

  - Noise can be modeled as an additive function, independent of image signal, to the convolution.

$$g(x,y) = f(x,y) * h(x,y) + n(x,y)$$

- In frequency domain:

$$G(u,v) = F(u,v)H(u,v) + N(u,v)$$

- Signal-to-noise ratio (SNR):

$$SNR = \frac{F}{E} = \frac{\sum_{(x,y)} n^2(x,y)}{\sum_{(x,y)} g^2(x,y)}$$

Noise in Images

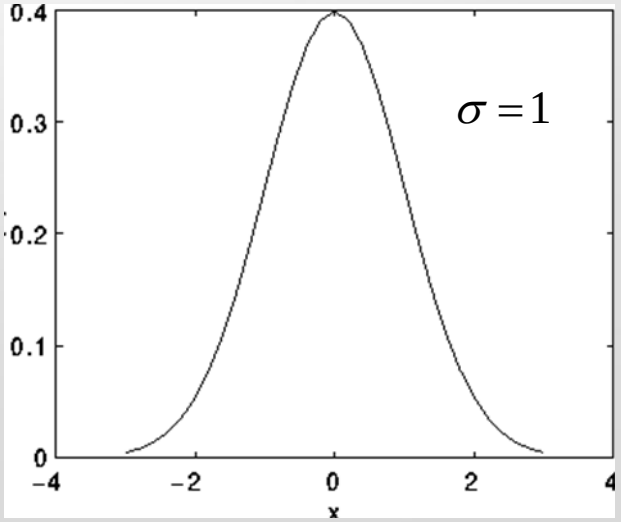
- Images are often degraded by random noise.
- Noise can occur during image capture, transmission or processing, and may be dependent on or independent of image content.
- Noise is usually described by its probabilistic characteristics.
- White noise -- constant power spectrum (its intensity does not decrease with increasing frequency);
- It is frequently applied as a crude approximation of image noise in most cases.
- The advantage is that it simplifies the calculations

Gaussian Noise

- A very good approximation of noise that occurs in many practical cases.
- Probability density of the random variable is given by the Gaussian function.
- 1D Gaussian noise -- μ the mean and σ is the standard deviation of the random variable.

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$

0 mean and std dev 1



Generation of Gaussian Noise

- Step1: Assuming o mean, Select value for σ , low value for less noise effect.
- Step2: For image gray-level i=0, 1, …, N-1 , calculate

$$p[i] = \frac{1}{\sigma\sqrt{2\pi}} e^{-i^2/2\sigma^2}$$

- Step3: For each pixel (x,y) with intensity f(x,y),

$$f^*(x,y) = f(x,y) + q_2 * j$$

- generate a random # q_1 in the range [0,1]. Determine

$$j = \operatorname{argmin}_i \left(\left| q_1 - p[i] \right| \right)$$

(cont)

- Step4: Generate a random # q_2 from {-1,1}. Set

$$f^*(x,y) = f(x,y) + q_2 * j$$

- Step5: set

$$f(x,y) = 0 \quad \text{if} \quad f^*(x,y) < 0$$

$$f(x,y) = G-1 \quad \text{if} \quad f^*(x,y) > G-1$$

$$f(x,y) = f^*(x,y) \quad \text{otherwise}$$

Salt and Pepper Noise

- Also called impulse noise, shot noise or binary noise.
- Appearance is randomly scattered white (salt) or black (pepper) pixels over the image.



Other Types of Noise

- Speckle noise: Multiplicative noise.

$$g = f + nf = f(1+n) \approx fn$$

  - May look superficially similar to Gaussian noise, but require different methods for removal.

- Periodic noise: Noise is periodic, rather than randomly distributed.

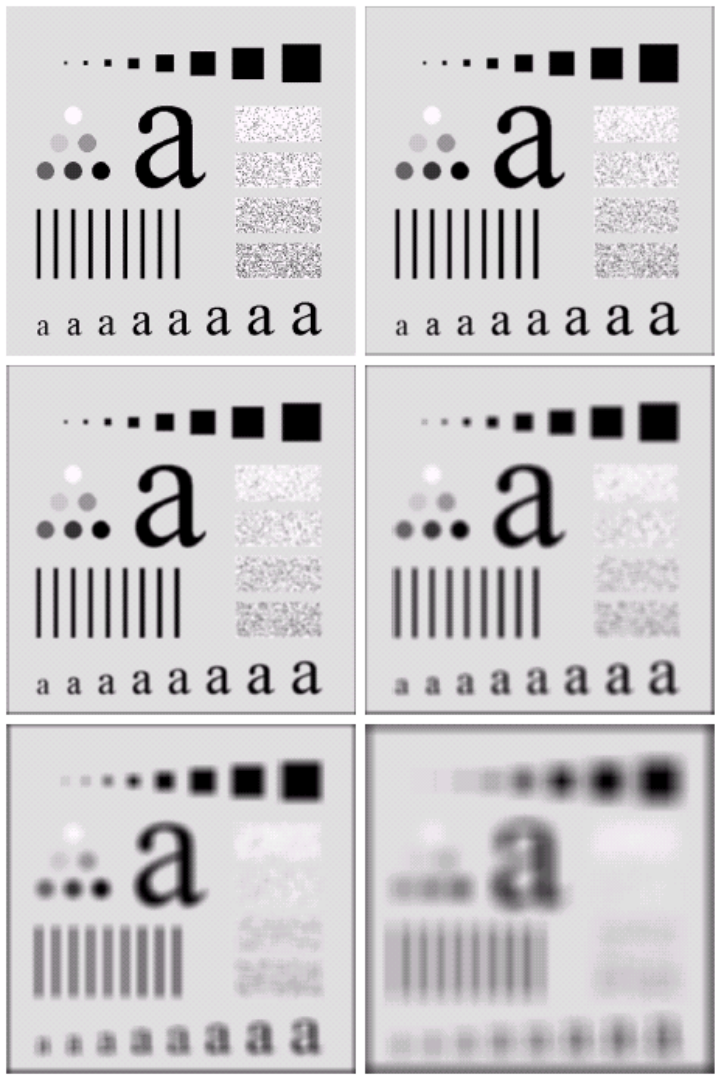


- Remark:
  - Only periodic noise is global effect. Others can be models as local degradation.
  - Spatial filtering is the method of choice for noise removal when the noise is additive.

Mean Filtering: Review

- Briefly discussed in lecture 4.
- The larger the kernel, the smoother/blurrier the image appears.
- Images on the right are produced using kernel of sizes

1	3
5	9
15	35



Mean Filtering

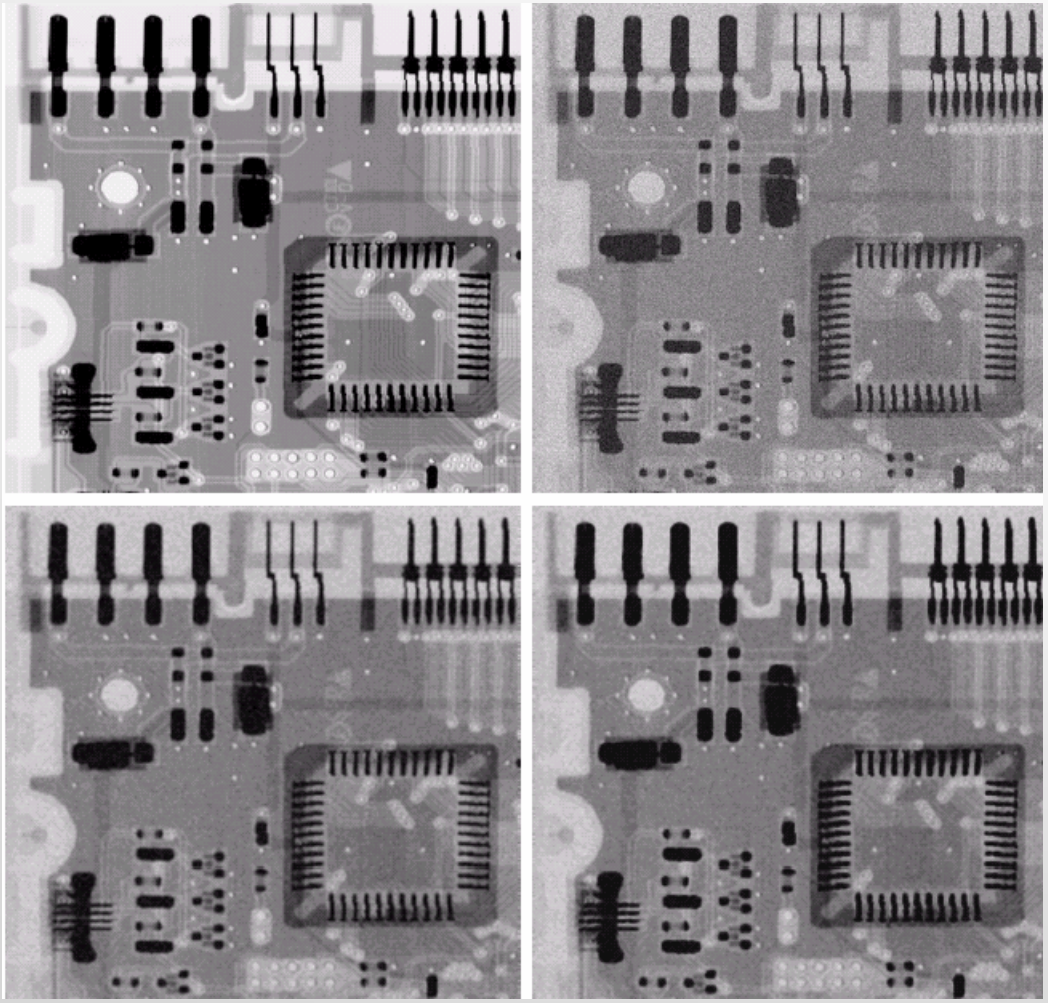
- Arithmetic:
  - average value of the corrupted image g(x,y) in the area defined by mask S of size *m* x *n*
  - The kernel contains coefficients of value 1/*mn*.
  - Smoothing local variations; noise reduction as a result of blurring.
- Geometric:

$$\hat{f}(x,y)=\left[\prod_{(a,b)\subseteq S} g(x-a,y-b)\right]^{\frac{1}{mn}}$$

  - Smoothing is comparable to arithmetic mean
  - Tend to lose less image detail.
- Q: What if there is one pixel with o intensity in the neighborhood?

Mean Filtering: Demo

Original



Corrupted by Gaussian noise

Arithmetic mean

Geometric mean

Mean Filter – Drawbacks

- What are the drawbacks with mean filtering?
  - A single pixel with a very unrepresentative value can significantly affect the mean value of all the pixels in its neighborhood.
  - When the filter neighborhood straddles an edge, the filter will interpolate new values for pixels on the edge and so will blur that edge. This may be a problem if sharp edges are required in the output.



Original



Salt and pepper noise



Mean filtering

Order-Statistics Filter

- The response is based on ordering the pixels contained in the image area encompassed by the filter.
- There are several variations:
  - Median:

$$\hat{f}(x,y) = \underset{(a,b)\subseteq S}{\text{median}} \{g(x-a, y-b)\}$$

  - Max:

$$\hat{f}(x,y) = \underset{(a,b)\subseteq S}{\max} \{g(x-a, y-b)\}$$

Order-Statistics Filter (con'd)

- Min:

$$\hat{f}(x,y) = \min_{(a,b)\subseteq S} \{g(x-a, y-b)\}$$

- Midpoint:

$$\hat{f}(x,y) = \frac{1}{2}\left[\max_{(a,b)\subseteq S} \{g(x-a, y-b)\} + \min_{(a,b)\subseteq S} \{g(x-a, y-b)\}\right]$$

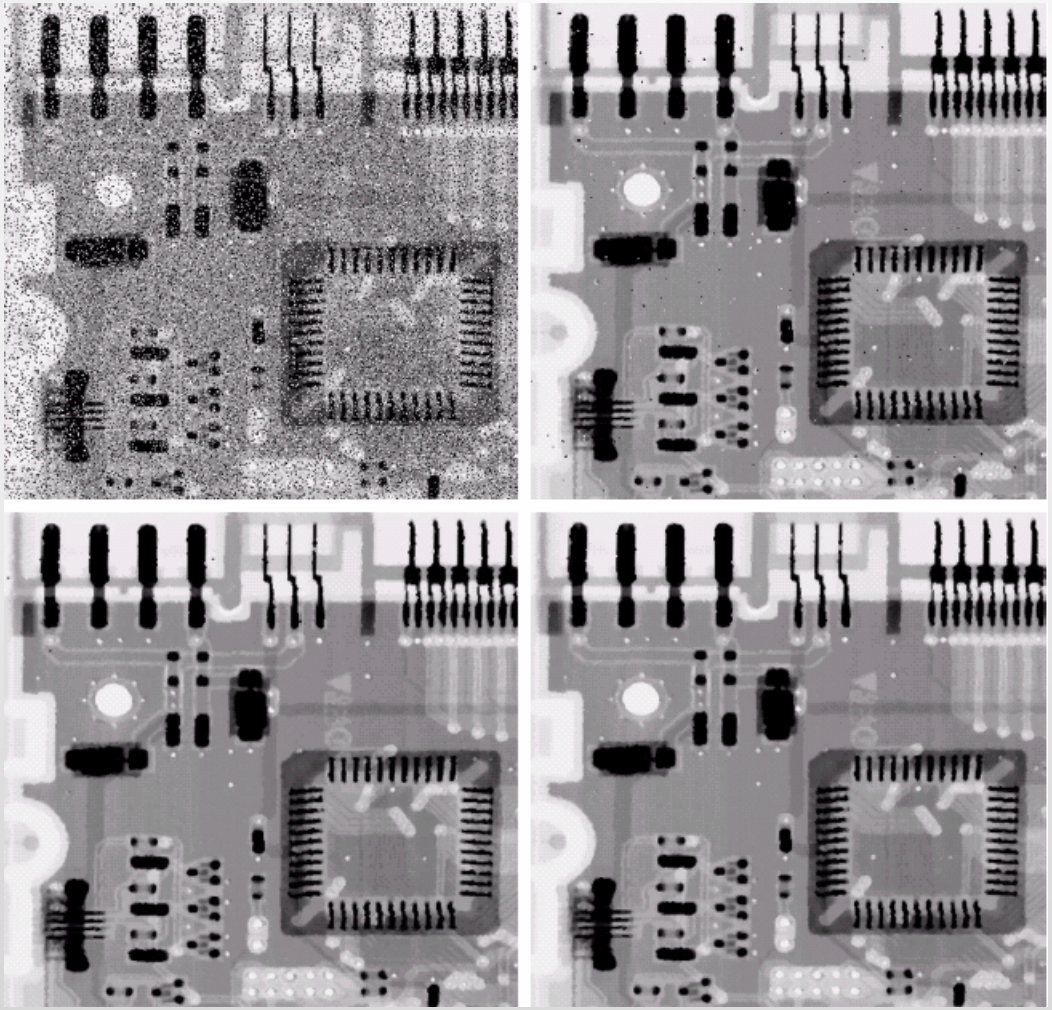
- Alpha-trimmed mean filter:
  - Delete the d/2 lowest and d/2 highest gray-level values of g(x-a,y-b)
  - Let $g_r(x,y)$ be the sum of the remaining pixels.

$$\hat{f}(x,y) = \frac{g_r(x,y)}{mn-d}$$

Order-Statistics Filter: Demo1

- Repeated application of the median filter

Corrupted by pepper-and-salt noise



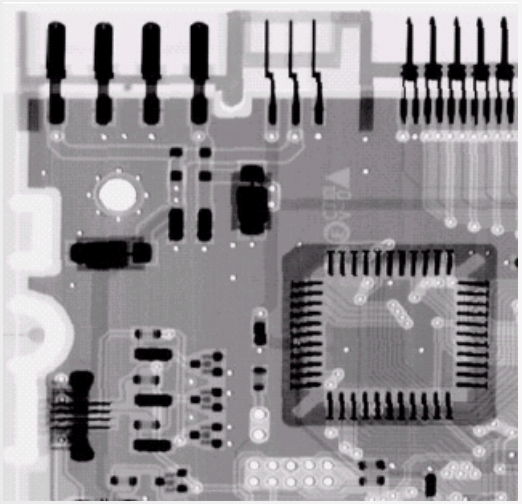
1st time

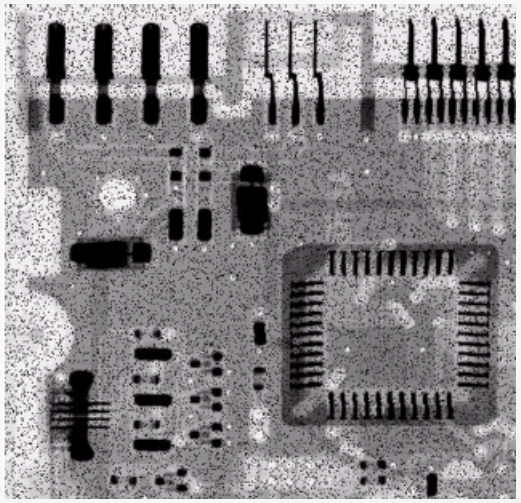
2nd time

3rd time

Order-Statistics Filter: Demo2

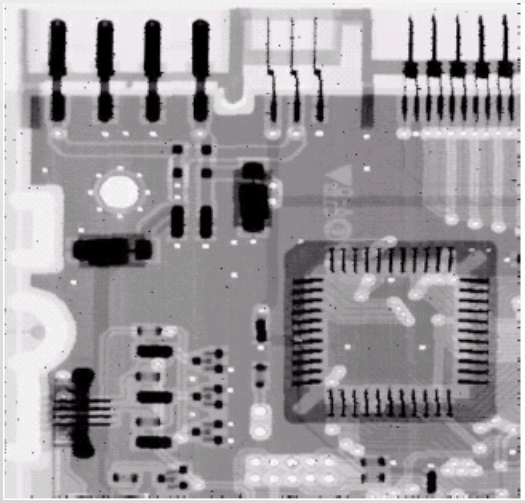
original

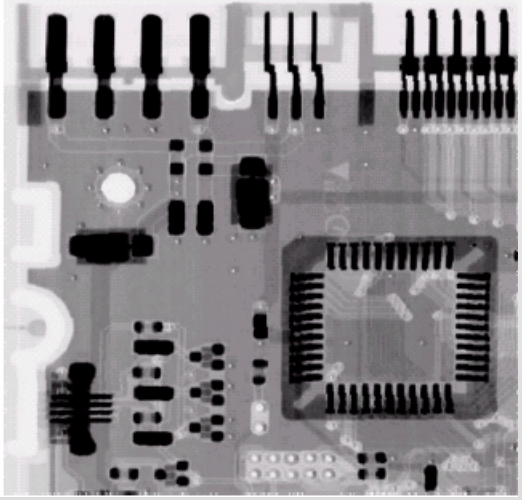




Corrupted by pepper noise

Max filter





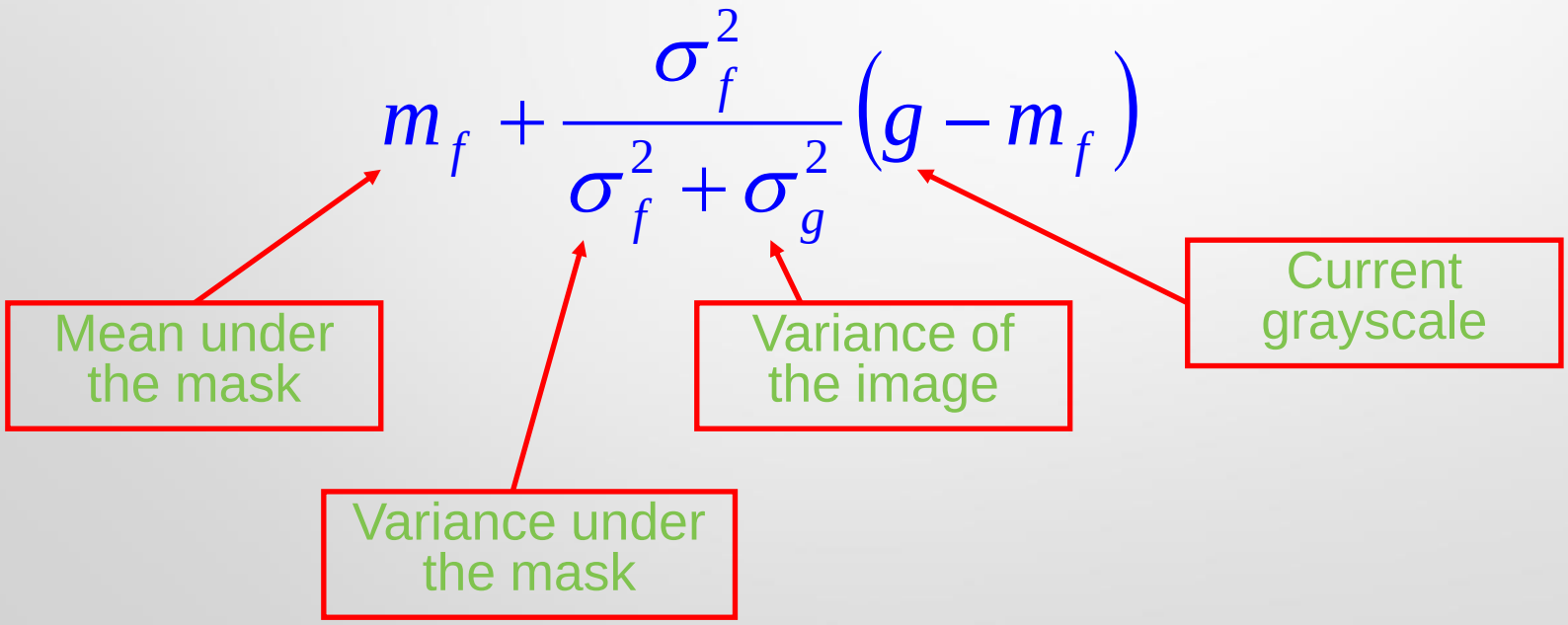
Min filter

Order-Statistics Filter: Drawback(s)

- Relatively expensive and complex to compute. To find the median it is necessary to sort all the values in the neighborhood into numerical order and this is relatively slow, even with fast sorting algorithms such as *quicksort*.
- Possible remedies?
  - When the neighborhood window is slid across the image, many of the pixels in the window are the same from one step to the next, and the relative ordering of these with each other will obviously not have changed.

Adaptive Filtering

- Changing the behavior according to the values of the grayscales under the mask.



Adaptive Filtering (con'd)

$$m_f + \frac{\sigma_f^2}{\sigma_f^2 + \sigma_g^2}\left(g - m_f\right)$$

- If σ_f^2 is high, then the fraction is close to 1; the output is close to the original value g.
  - High σ_f^2 implies significant detail, such as edges.
- If the local variance is low, such as the backgrou the fraction is close to 0; the output is close to m_f

Adaptive Filtering: Variation

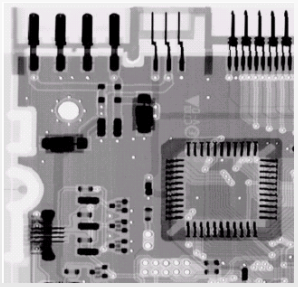
- σ_g^2 is often unknown, so is taken as the mean of all values of σ_f^2 over the entire image.
- In practice, we adopt the slight variant:

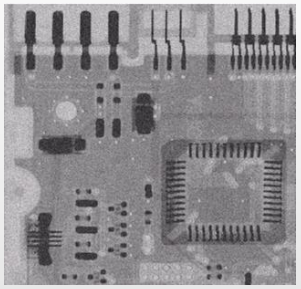
$$m_f + \frac{\max\left\{0, \sigma_f^2 - \sigma_g^2\right\}}{\max\left\{\sigma_f^2, \sigma_g^2\right\}}\left(g - m_f\right)$$

- In Matlab, the function is named "wiener2".

Adaptive Filtering: Demo (7x7 mask)

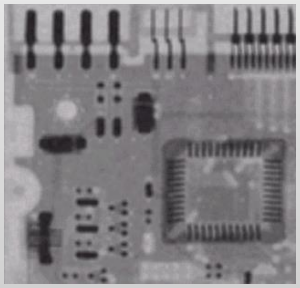
original

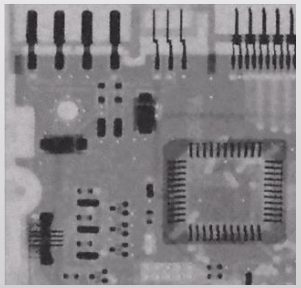




Corrupted by Gaussian noise with variance=1000

Mean filter





Adaptive filtering

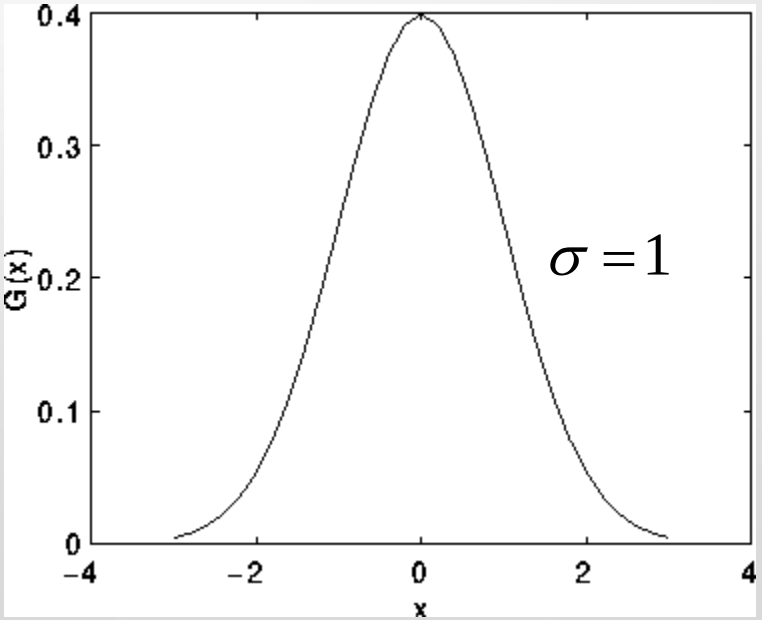
Gaussian Smoothing

- The Gaussian smoothing operator is used to `blur' images and remove detail and noise. In this sense it is similar to the mean filter, but it uses a different kernel that represents the shape of a Gaussian (`bell-shaped') hump.
- In 1-D, 0 mean and

  std dev : σ

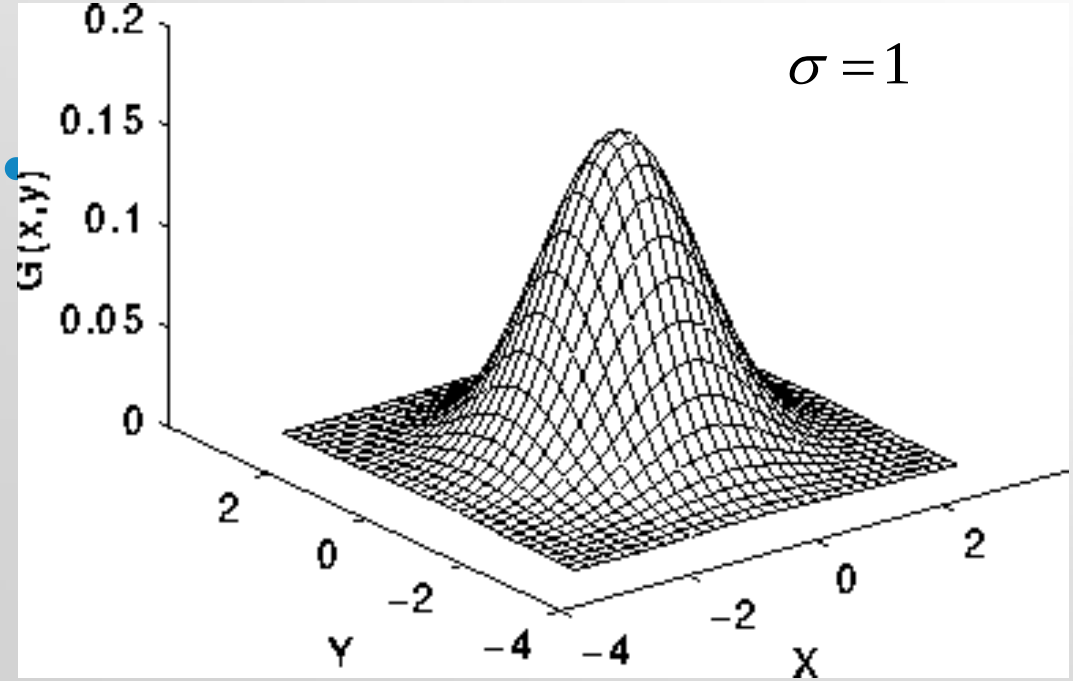
$$G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

We have seen this in filtering in frequency domain



Gaussian Smoothing

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$



- The Gaussian outputs a `weighted average' of each pixel's neighbourhood, with the average weighted more towards the value of the central pixels.

Gaussian Smoothing

- The only parameter is σ(sigma), which is the standard deviation. This value controls the degree of smoothing.
  - as this parameter goes up, more pixels in the neighborhood are involved in “averaging”,
  - the image gets more blurred, and
  - noise is more effectively suppressed

How to discretize the Gaussian function?

- In theory, the Gaussian distribution is non-zero everywhere, which would require an infinitely large convolution mask.
- but in practice it is effectively zero more than about three from the mean, and so we can truncate the mask at this point.

σ

$$\frac{1}{115}$$

2	4	5	4	2
4	9	12	9	4
5	12	15	12	5
4	9	12	9	4
2	4	5	4	2

Discrete approximation to Gaussian function with $\sigma = 1.4$

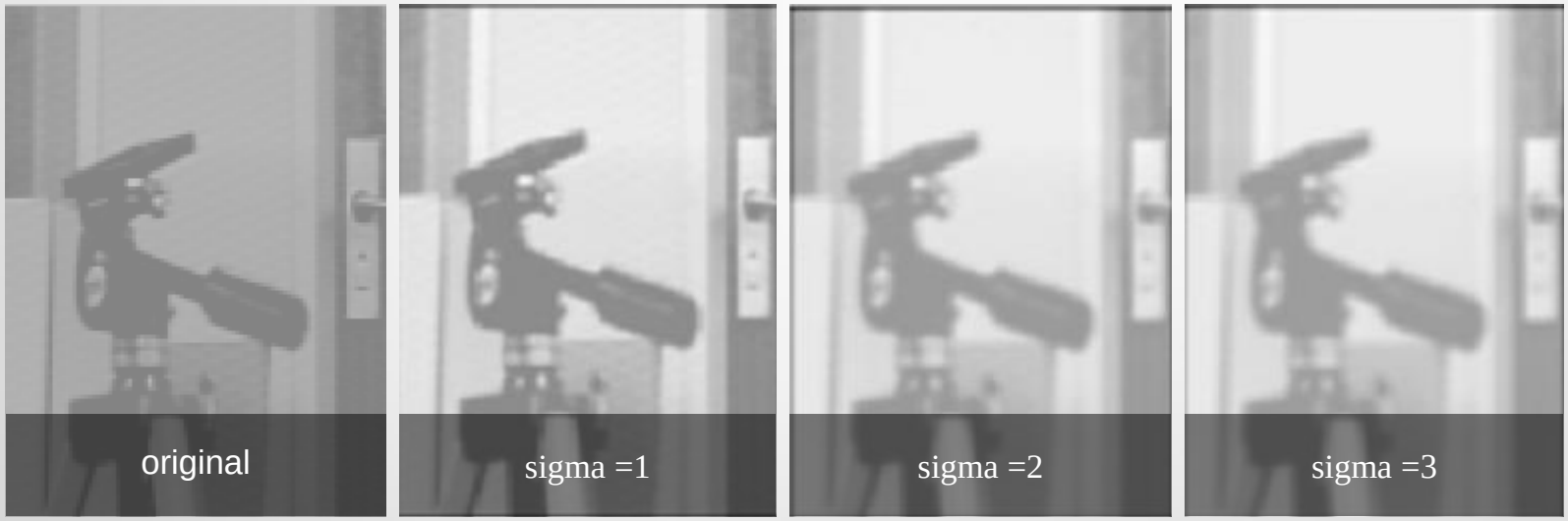
Separable Gaussian Kernel

- 2-D isotropic Gaussian is separable into *x* and *y* components.
- Thus the 2-D convolution can be performed by first convolving with a 1-D Gaussian in the *x* direction, and then convolving with another 1-D Gaussian in the *y* direction.
- 1-D *x* component for the 2D kernel.

$$\frac{1}{10.7}\begin{bmatrix} 1.3 & 3.2 & 3.8 & 3.2 & 1.3 \end{bmatrix}$$

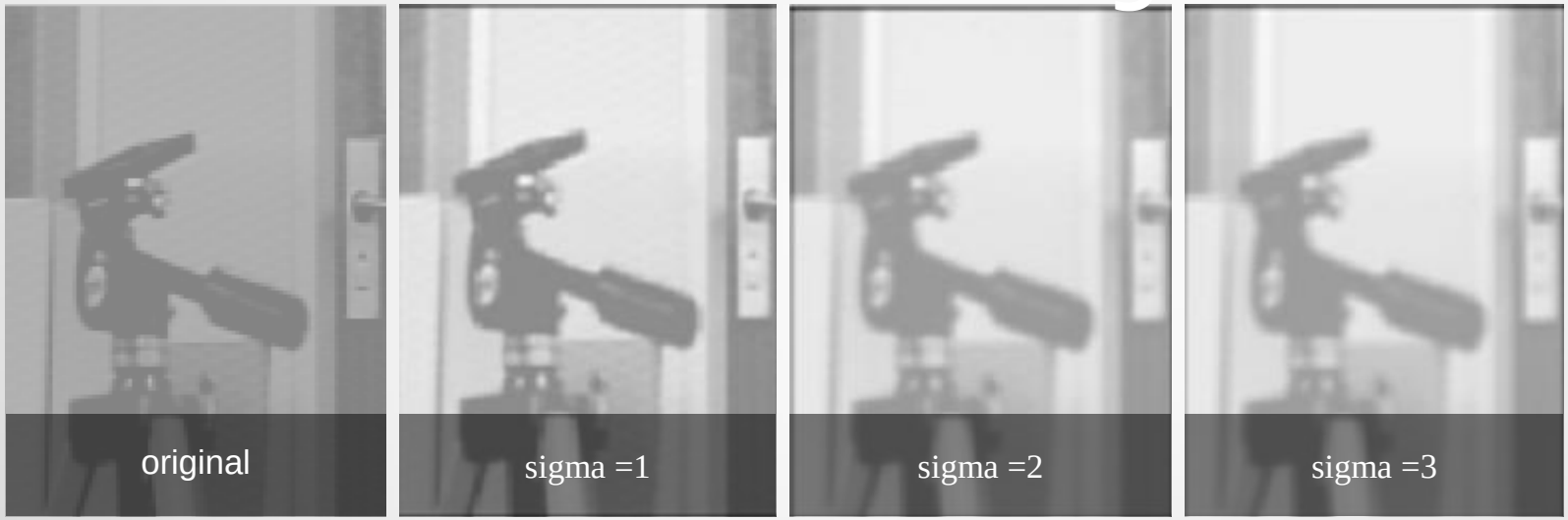
- The *y* component is exactly the same but is oriented vertically.

Gaussian Smoothing



The difference in the output image is stronger around edges in the image. For a homogenous area, Gaussian filters with different sigma values give almost the same result, but along edges the result differs a lot.

Gaussian Smoothing



sigma as setting the scale at which we preserve information in the convolved image. Structure on a scale small compared with
sigma will be removed, whilst structure on a larger scale is retained.

Gaussian filters with different sigma values give different smoothing. The difference in the output image is stronger around edges in the image. For a homogenous area Gaussian filters with different sigma values give almost the same result, but along egdes the result differs a lot. When subtracting images with different sigma values homogenous areas are black (the results subtract to zero) and edges are white (high pixel values).